

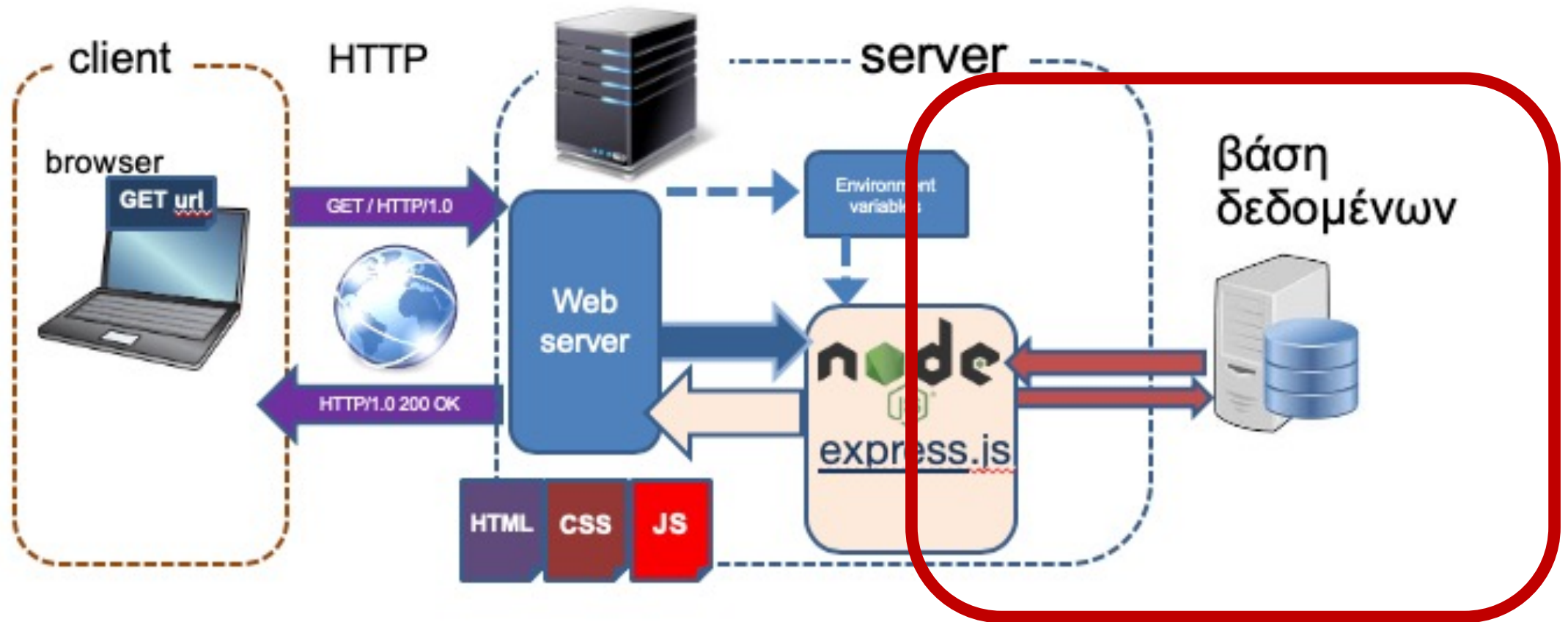
ECE_ΓΚ802 ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ ΔΙΑΔΙΚΤΥΟΥ

8^ο εξάμηνο

Ενότητα 05 (εβδομάδες 10-12)

Προγραμματισμός στην πλευρά
του εξυπηρετητή – σύνδεση με
βάση δεδομένων

Σύνδεση με μόνιμη αποθήκευση δεδομένων



περιεχόμενα

- Ο κύκλος ζωής ανάλυσης δεδομένων μιας εφαρμογής – παράδειγμα
- Το μοντέλο MVC (model-view-controller)
- Σχεδίαση της διεπαφής του model
- Υλοποίηση μόνιμης αποθήκευσης
 - Αρχεία JSON
 - Βάση δεδομένων SQLITE3
 - Βάση δεδομένων PostgreSQL
- Άλλες βάσεις δεδομένων: MySQL, Mongo/Mongoose

Ο μικρόκοσμος: Τα βιβλία της παρέας μας

Μια παρέα
(πολλοί χρήστες),
αποθηκεύουν και
σχολιάζουν τα
βιβλία τους

 myBooks

βιβλία...
σχετικά..

myBooks ...
τα βιβλία της παρέας μας

δώστε το όνομά σας...

Υποβολή

© ECE_ΓΚ802 - myBooks

myBooks: Βασική οθόνη

 myBooks βιβλία... σχετικά...

Χρήστης

Εισαγωγή/
Διαγραφή/
Τροποποίηση
Βιβλίων

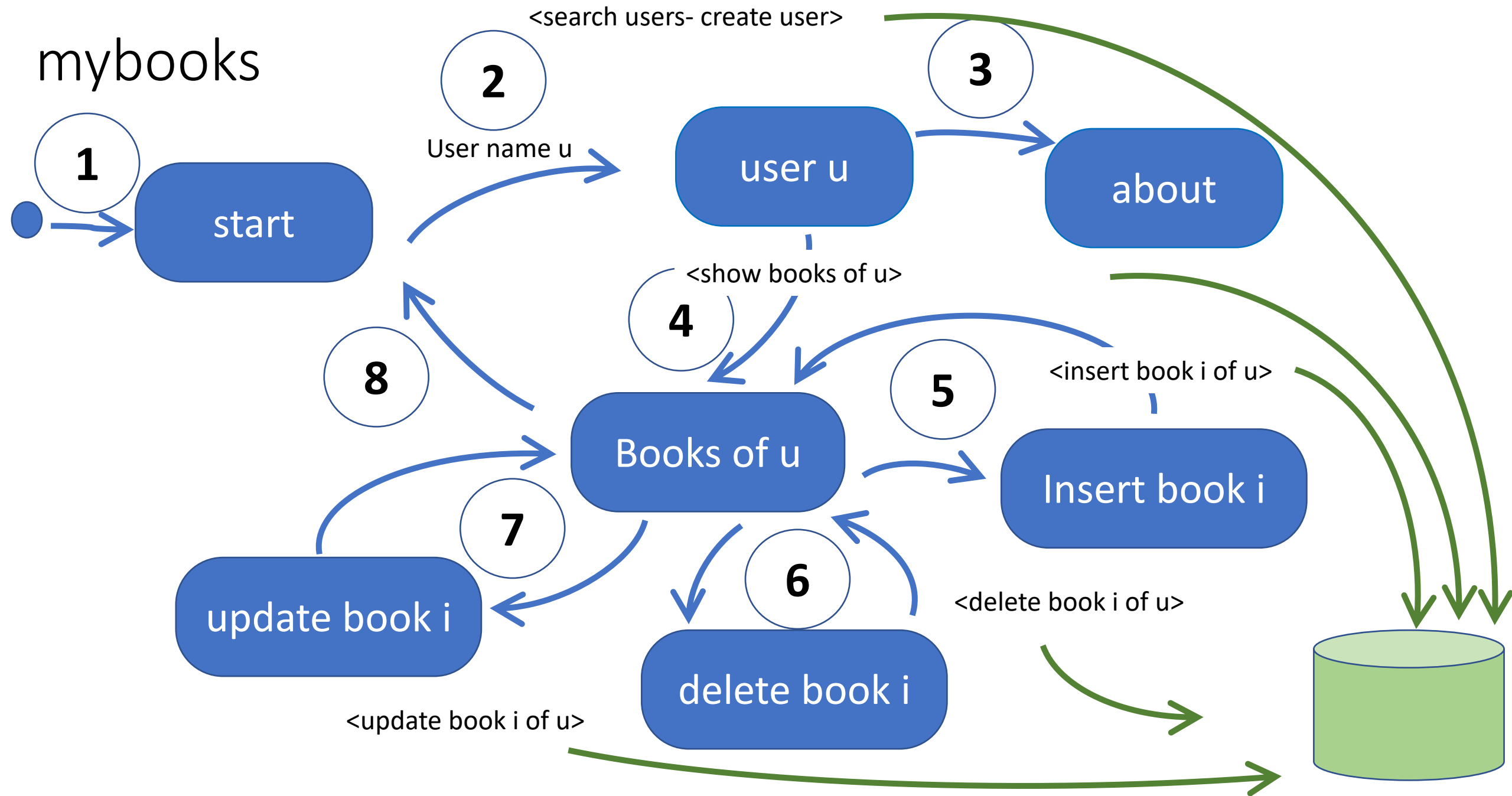
τα βιβλία μου - Nikos (3)

Τίτλος	Συγγραφείς	Σχόλια	Προσθήκη
Theory and Design of Pervasive Games	Markus Montola	βασικό βιβλίο για χωρικά παιχνίδια, highly recommended	Αλλαγή Διαγραφή
Τεχνοπώλιο	Niel Postman	εξαιρετική κριτική στα σύγχρονα μέσα ενημέρωσης	Αλλαγή Διαγραφή
ένα ακόμη	Συγγραφέας	δεν έχω πολλά να πω για το βιβλίο τούτο	Αλλαγή Διαγραφή

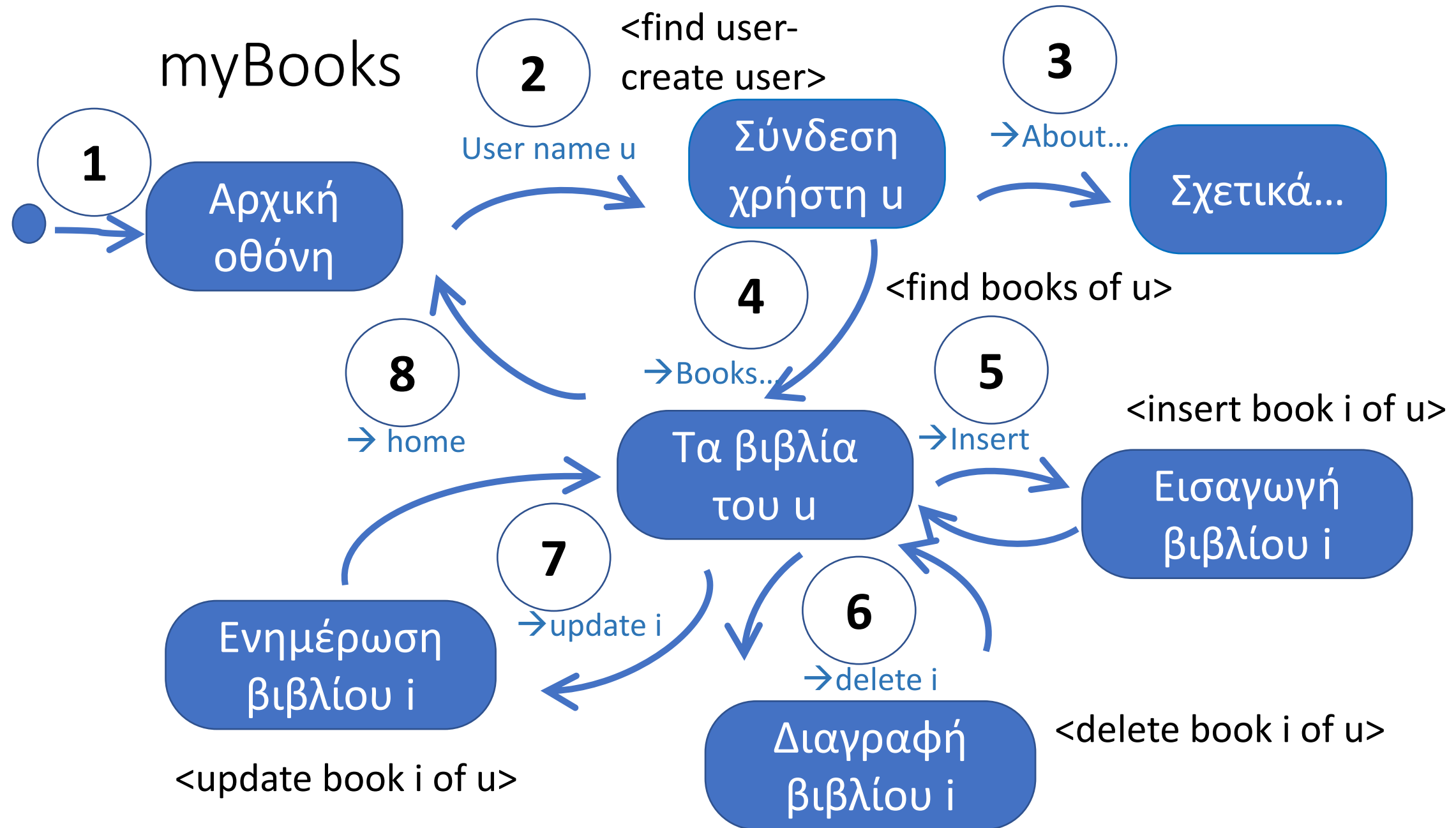
© ECE_ΓK802 - myBooks



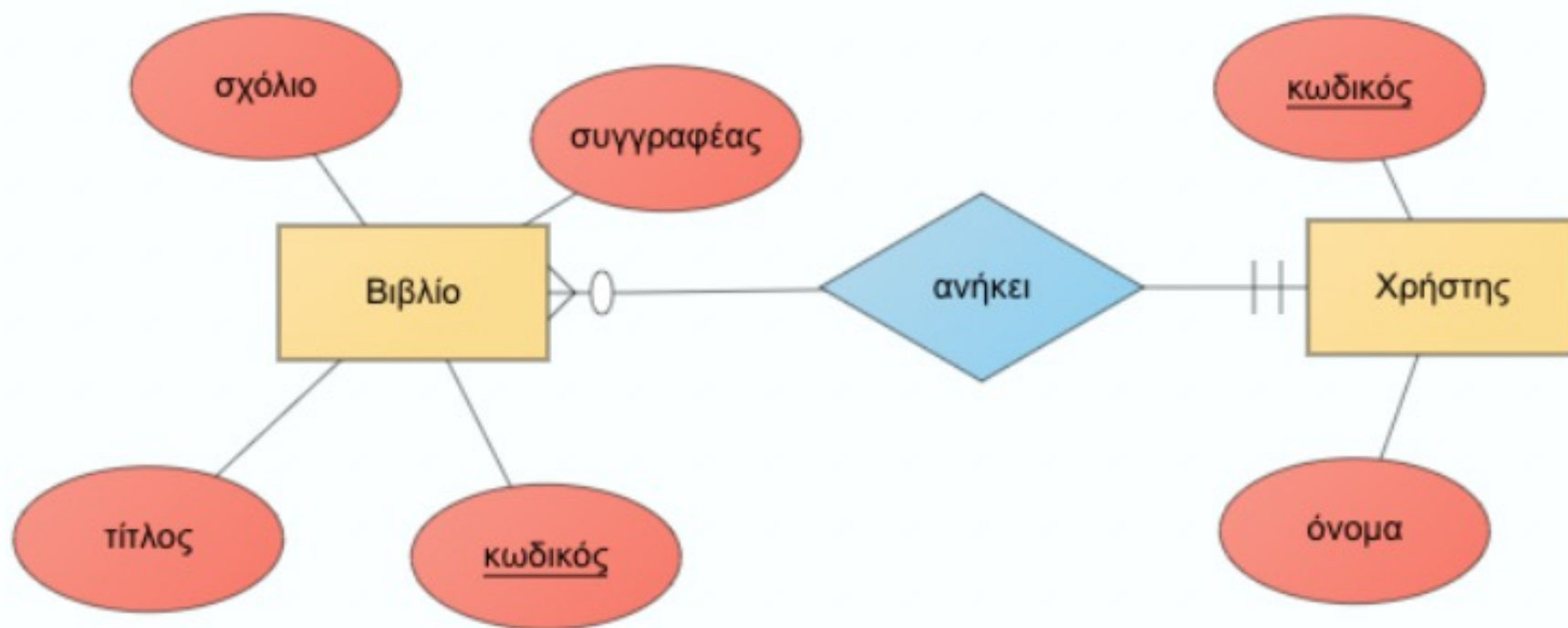
mybooks



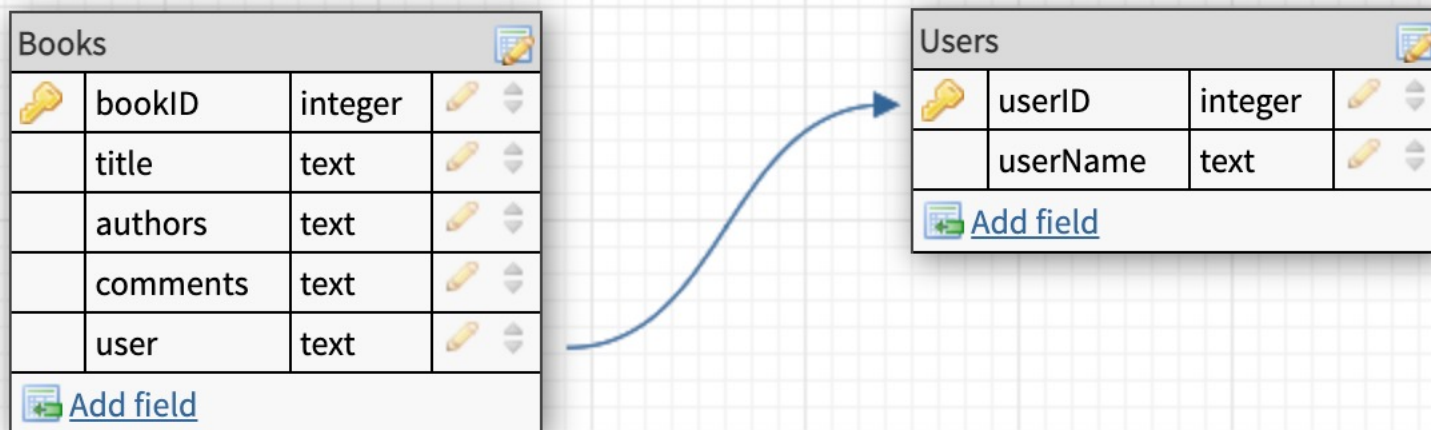
myBooks



Διάγραμμα οντοτήτων-συσχετίσεων



Σχεσιακό μοντέλο. - διεπαφή με τη βδ

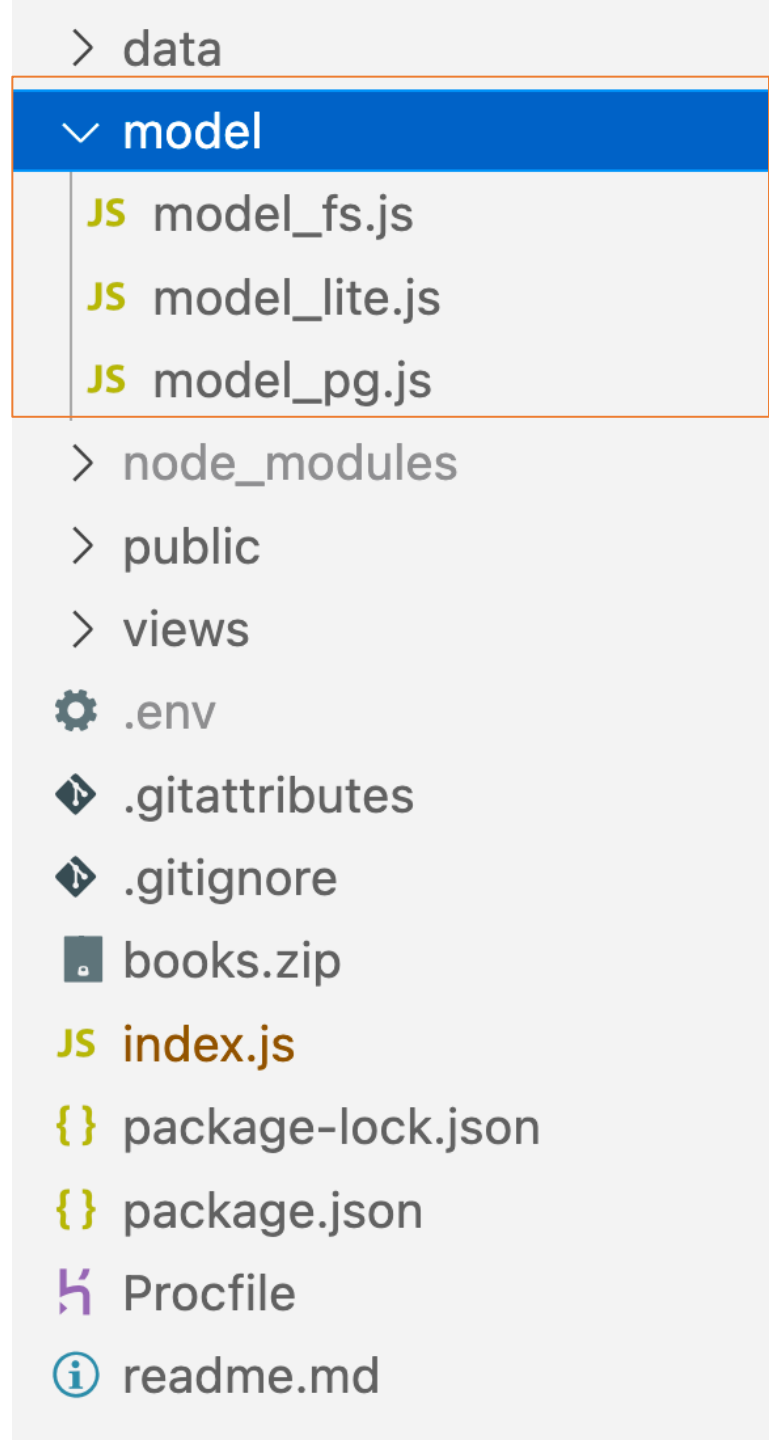


getMyBooks,
newBook,
findBook,
updateBook,
deleteBook,

insertUser,
findUser

Σχεδίαση της ΒΔ και της διεπαφής

- Ακολουθούμε το μοντέλο Model-View-Controller
- Συγκεντρώνουμε στο Model τη διεπαφή (API) με τη ΒΔ
- Υλοποιούμε το μοντέλο για διαφορετικές ΒΔ:
 - **JSON** file (import fs)
 - **SQLite3** βάση δεδομένων (import sqlite3)
 - **Postgresql** βάση δεδομένων (import pg)
 - **Mongo** – mongoose (import mongoose)

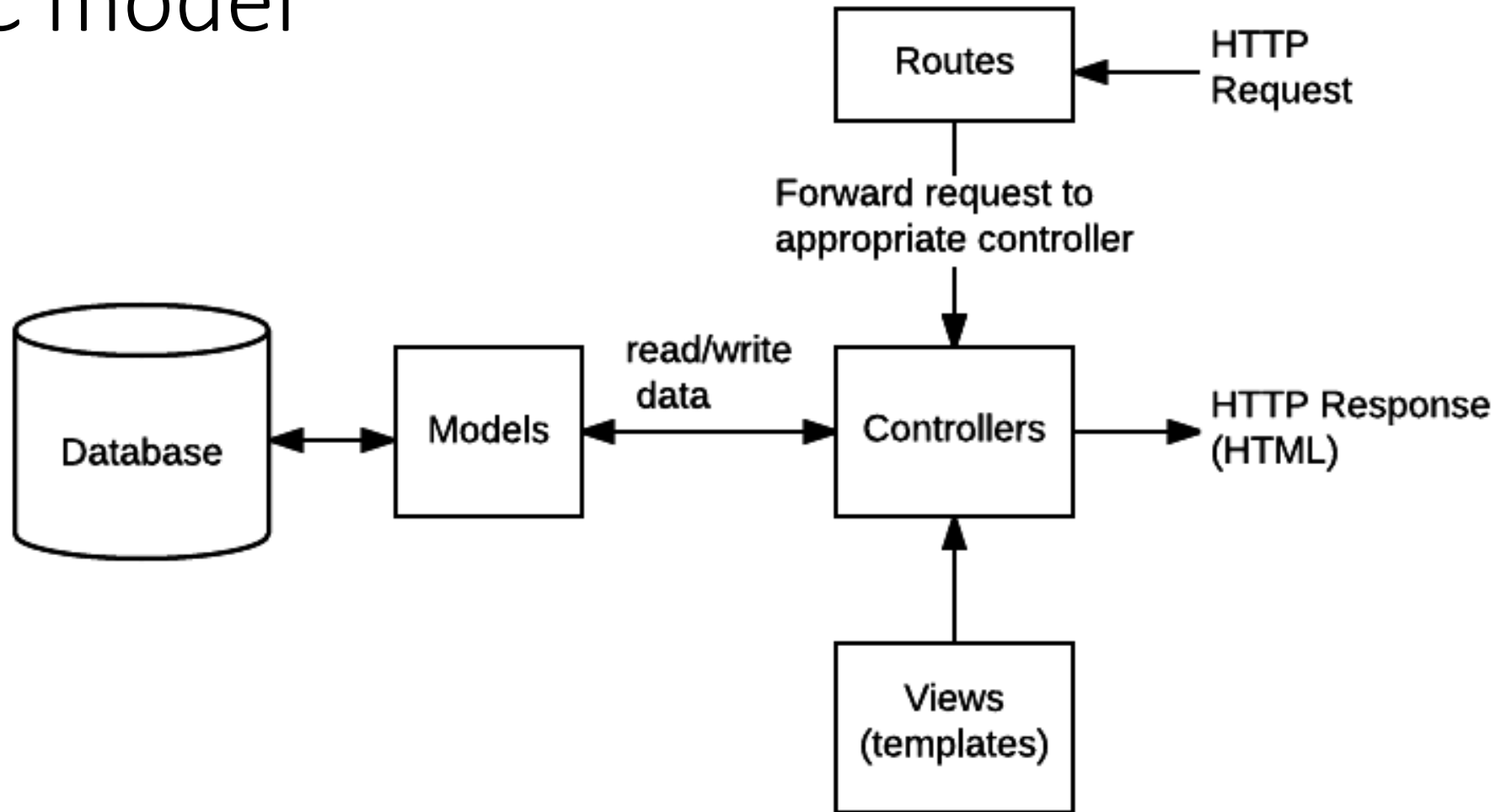


Project structure

```
— controllers/
  — mongo/
    — about.mjs
    — create.mjs
    — createBook.mjs
    — deleteBook.mjs
    — findBook.mjs
    — findToDeleteBook.mjs
    — getBooks.mjs
    — getCreateUser.mjs
    — getUser.mjs
    — updateBook.mjs
  — sqlite/
    — about.mjs
    — create.mjs
    — createBook.mjs
    — deleteBook.mjs
    — findBook.mjs
    — findToDeleteBook.mjs
    — getBooks.mjs
    — getCreateUser.mjs
    — getUser.mjs
    — updateBook.mjs
  — data/
    — books-pg.sql
    — books.db
    — books.json
    — books.sql
    — users.json
  — public/
```

```
— about.mjs
— create.mjs
— createBook.mjs
— deleteBook.mjs
— findBook.mjs
— findToDeleteBook.mjs
— getBooks.mjs
— getCreateUser.mjs
— getUser.mjs
— updateBook.mjs
— data/
  — books-pg.sql
  — books.db
  — books.json
  — books.sql
  — users.json
— public/
  — .DS_Store
  — css/
    — bootstrap.min.css
  — model/
  — mybooks.png
— routes/
  — router.mjs
— views/
  — about.hbs
  — books.hbs
  — create.hbs
  — delete.hbs
  — edit.hbs
  — index.hbs
  — layouts/
    — main.hbs
— index.js
— package.json
— readme.md
```

MVC model



Η διεπαφή model για την εφαρμογή μας

```
getMyBooks = (userID, callback) => {  
  /// read books of user userID  
}
```

```
newBook = (book, callback) => {  
  /// insert book in database  
}
```

```
findBook = (bookID, callback) => {  
  /// find book with bookID  
}
```

```
updateBook = (book, callback) => {  
  /// update book in database  
}
```

```
deleteBook = (bookID, callback) => {  
  /// delete book from database  
}
```

```
insertUser = (user, callback) => {  
  // insert user and return the inserted  
  // user in the callback  
}
```

```
findUser = (userID=null, userName=null,  
  callback) => {  
  // find user with userName if it does not  
  // exist create new user, find user with  
  // userID  
}
```

Η συνάρτηση `callback(err, result)=> {}` ορίζεται κατά την κλήση της διεπαφής και χειρίζεται σφάλμα αν υπάρχει ή τα δεδομένα που επιστρέφει η διεπαφή

πχ. `model.findUser(userID, userName, callback)`

```
app.post("/", (req, res) => {  
  // έχει συμπληρωθεί το userName στη φόρμα  
  // βρες τον χρήστη id ή δημιούργησε χρήστη αν δεν υπάρχει  
  let userID = null;  
  let userName = req.body.userName  
  model.findUser(userID, userName, (err, row) => {  
    console.log('POST / returned row....', row)  
    if (err){  
      console.log(err.message);  
    } else {  
      req.session.userID = row[0].userID;  
      req.session.userName = row[0].userName;  
      console.log("new session", req.session)  
    }  
    res.redirect("/")  
  });  
});
```

Υλοποίηση της μόνιμης αποθήκευσης σε
αρχεία JSON (model_fs.js)

Μόνιμη αποθήκευση σε αρχεία JSON

- Η λύση αυτή δεν ταιριάζει σε πραγματικές εφαρμογές, αφού το άνοιγμα/κλείσιμο αρχείων είναι δαπανηρή διαδικασία.
- Διάφορες πλατφόρμες νέφους platform as a service (PaaS), όπως η Heroku δεν υποστηρίζουν διατήρηση αλλαγών σε αρχεία του δίσκου, άρα οι αλλαγές χάνονται κάθε φορά που ένα dyno πέφτει σε νάρκη.

Μόνιμη αποθήκευση: δύο αρχεία JSON

books.json

```
[ {  
  "title": "Άπαντα",  
  "author": "Σεφέρη",  
  "comment": "πολλά καλά ποιήματα",  
  "bookID": "6",  
  "user": "1"  
}, ... ]
```

users.json

```
[ {  
  "userID": "1",  
  "userName": "Κατερίνα"  
},  
{  
  "userID": "2",  
  "userName": "Νίκος"  
}, ... ]
```

findBook

```
import fs from "fs";
```

```
const findBook = (bookID, callback) => {  
  console.log('findbook', bookID)  
  fs.readFile(db_name, {encoding:"utf-8"}, (err, data) => {  
    if (err) {  
      callback(err, null);  
    }  
    else {  
      const theBooks = JSON.parse(data)  
      const rows = theBooks.filter((el)=> el.bookID == bookID)  
      callback(null, rows)  
    }  
  })  
}
```

```
export {getMyBooks, newBook, findBook, updateBook, deleteBook, insertUser,  
findUser};
```

Υλοποίηση της μόνιμης αποθήκευσης με
βάση δεδομένων SQLite3 (model_lite.js)

SQLite

- Η SQLite είναι μια **ενσωματωμένη μηχανή βάσης δεδομένων SQL**.
- Σε αντίθεση με τις περισσότερες άλλες βάσεις δεδομένων SQL, η SQLite δεν διαθέτει ξεχωριστή διεργασία εξυπηρετητή.
- Η SQLite διαβάζει και γράφει απευθείας σε συνηθισμένα αρχεία στο δίσκο. Μια πλήρης βάση δεδομένων SQL με πολλαπλούς πίνακες, δείκτες, triggers και views, περιέχεται σε ένα μόνο αρχείο δίσκου.
- Η μορφή αρχείου της βάσης δεδομένων είναι ανεξάρτητη λειτουργικού συστήματος
- Διάφορες πλατφόρμες νέφους platform as a service (PaaS), όπως η Heroku δεν υποστηρίζουν βάσεις δεδομένων SQLite για διαδικτυακές εφαρμογές.

Σύμφωνα με το <https://www.sqlite.org> :

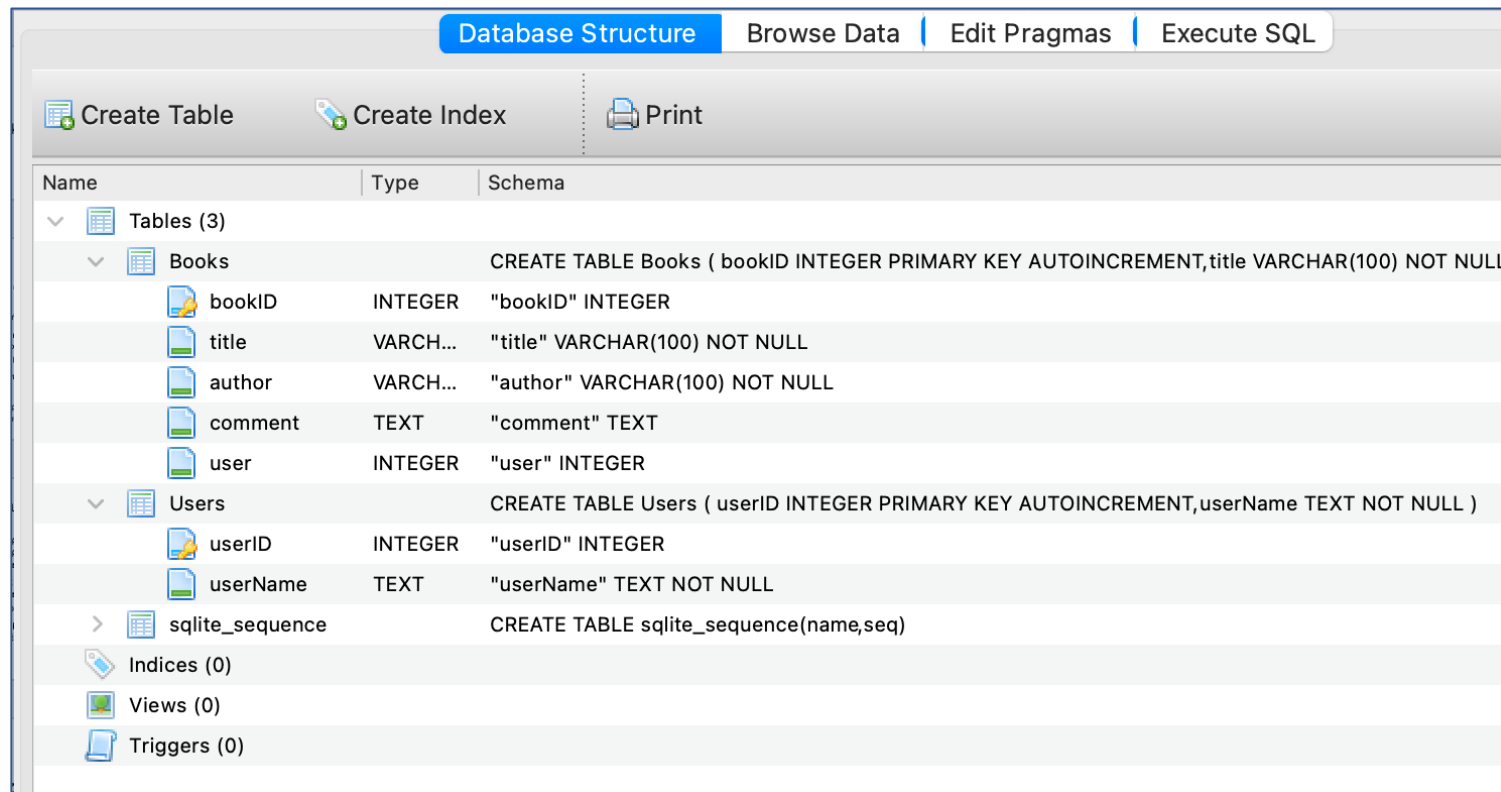
- Η SQLite λειτουργεί ικανοποιητικά ως μηχανή βάσης δεδομένων για τους περισσότερους ιστότοπους χαμηλής έως μέτριας επισκεψιμότητας (δηλαδή για τους περισσότερους ιστότοπους).
- Η ποσότητα της διαδικτυακής κίνησης που μπορεί να διαχειριστεί η SQLite εξαρτάται από το πόσο έντονα χρησιμοποιεί η ιστοσελίδα τη βάση δεδομένων της.
- Σε γενικές γραμμές, κάθε ιστότοπος που έχει λιγότερες από **100 χιλιάδες επισκέψεις/ημέρα** θα πρέπει να λειτουργεί ικανοποιητικά με την SQLite.

Sqlite3 API

- **db.run(sql [, param, ...] [, callback]);** // εκτελεί την εντολή sql, και δεν επιστρέφει δεδομένα
- **db.get(sql [, param, ...] [, callback])** // επιστρέφει την πρώτη εγγραφή στη συνάρτηση callback
- **db.all(sql [, param, ...] [, callback])** // επιστρέφει όλες τις εγγραφές στη συνάρτηση callback
- **db.close([callback]);** // κλείνει τη σύνδεση με τη βάση δεδομένων

Δημιουργούμε τη βάση δεδομένων books

- Χρήση γραφικού εργαλείου διαχείρισης db browser for sqlite ή επέκτασης του VS Code
- Αποθηκεύεται ως αρχείο /data/books.db



findBook

```
import sqlite3 from "sqlite3";
```

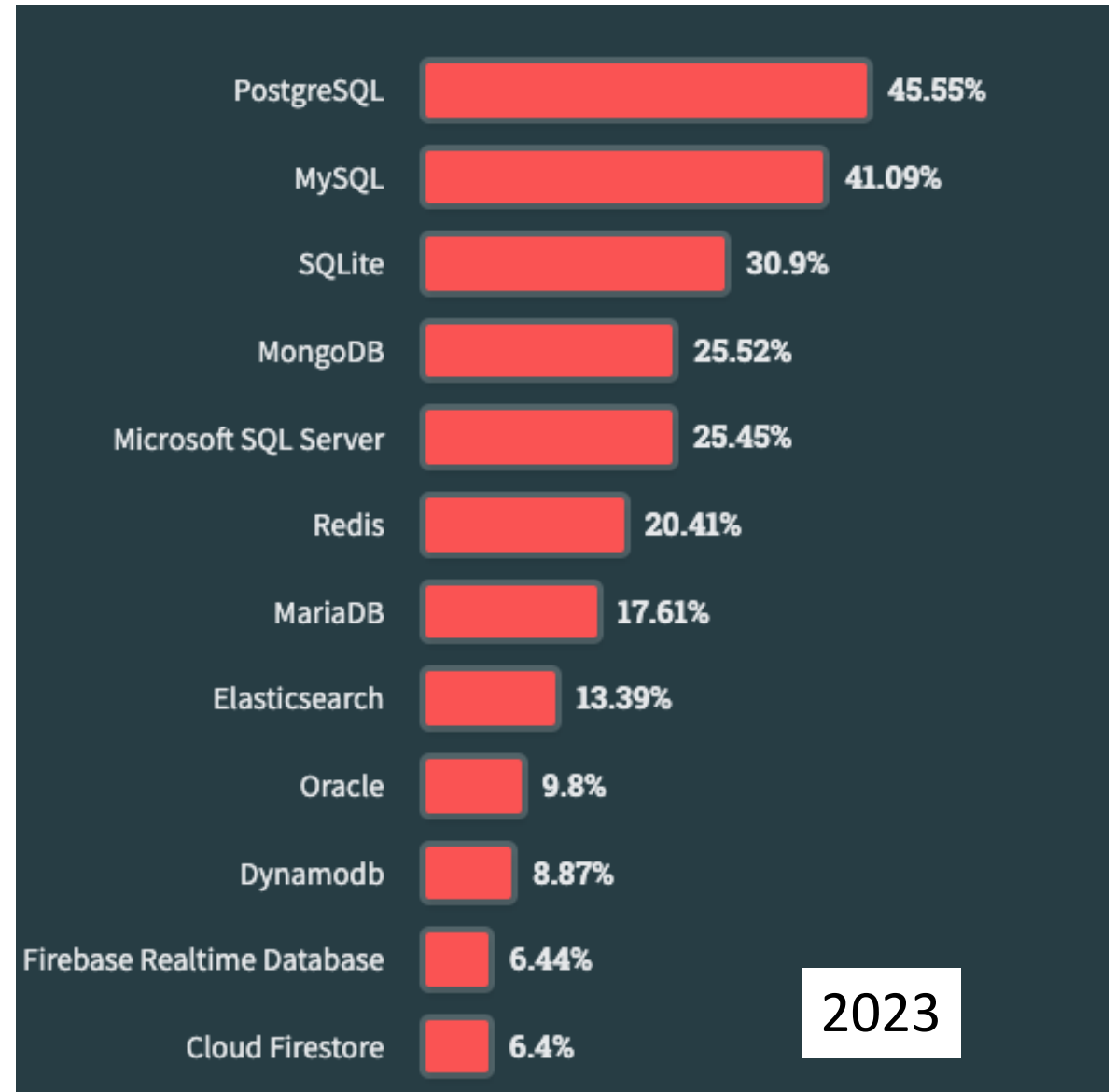
```
const findBook = (bookID, callback) => {  
  const sql = "SELECT * FROM Books WHERE bookID = ?";  
  const db = new sqlite3.Database(db_name);  
  db.get(sql, [bookID], (err, row) => {  
    db.close();  
    if (err) {  
      callback(err, null)}  
    else {  
      callback(null, [row])}  
    });  
}
```

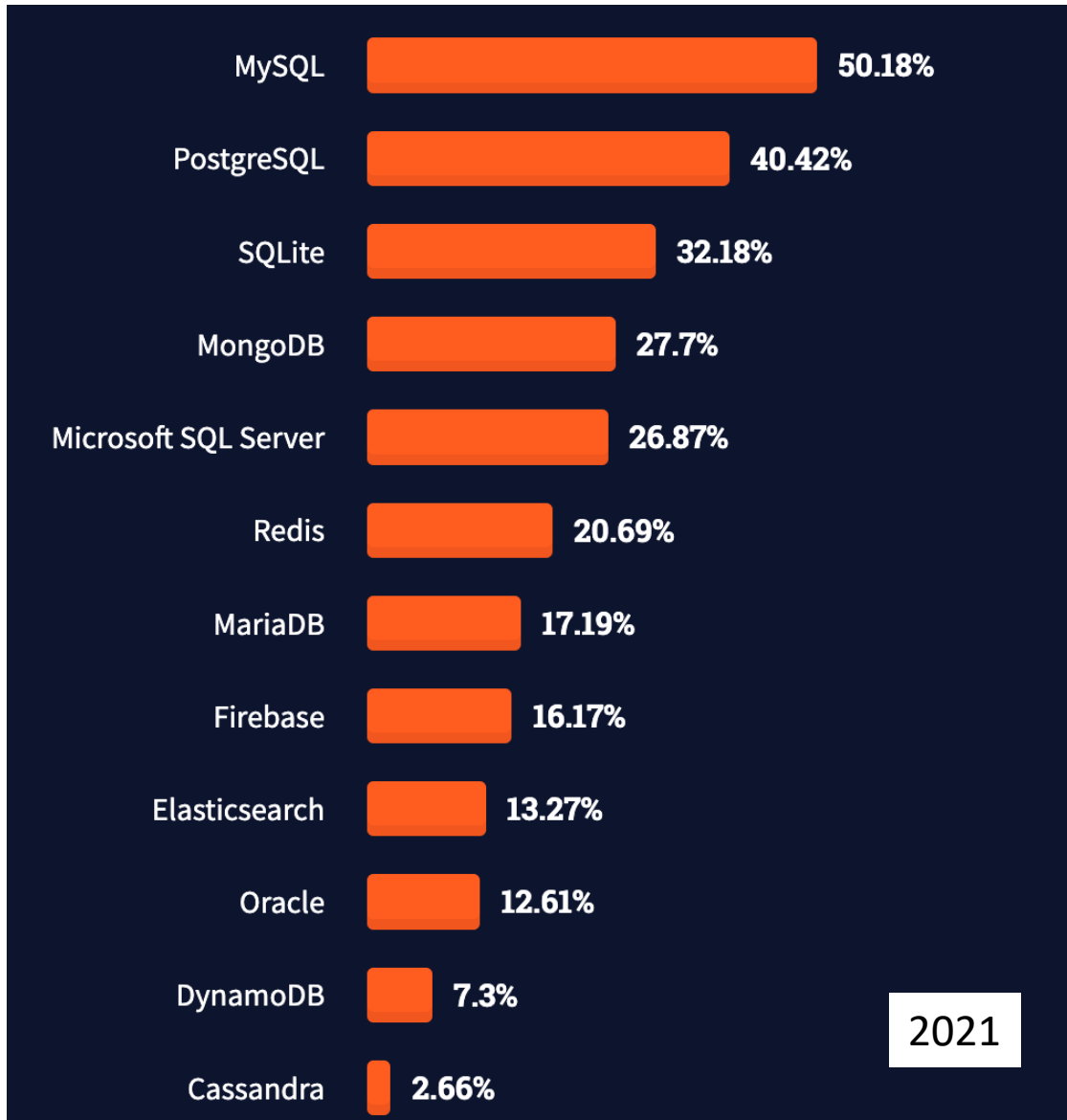
```
export {getMyBooks, newBook, findBook, updateBook, deleteBook, insertUser,  
findUser};
```


Υλοποίηση της μόνιμης αποθήκευσης με
βάση δεδομένων PostgreSQL (model_pg.js)

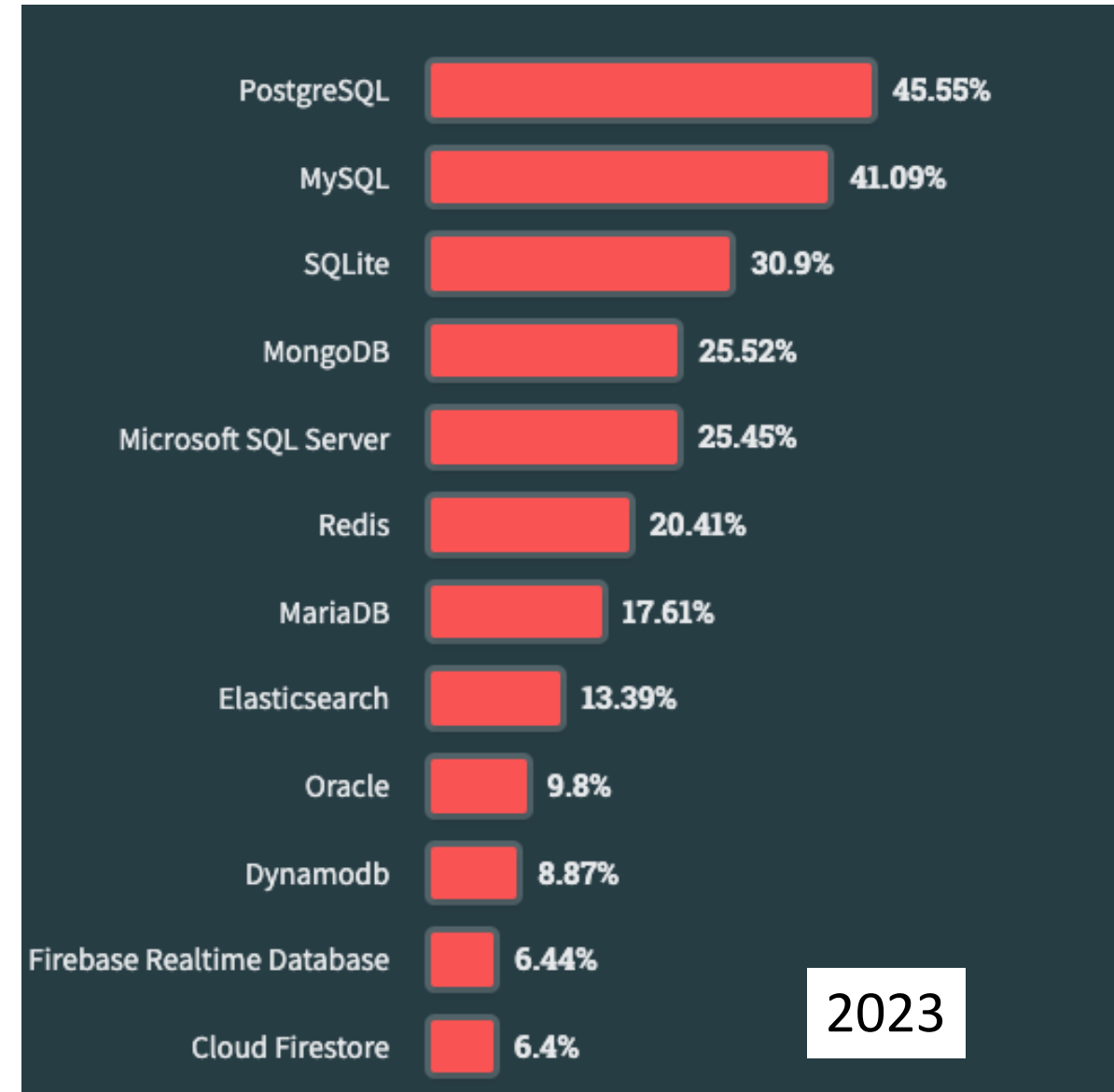
PostgreSQL

- Η PostgreSQL επίσης γνωστή ως Postgres, είναι ένα ελεύθερο και ανοικτού κώδικα σύστημα διαχείρισης σχεσιακών βάσεων δεδομένων (RDBMS) με έμφαση στην επεκτασιμότητα και τη συμμόρφωση με την SQL.
- Διαθέτει ισχυρό υποσύστημα για διαχείριση χωρικών δεδομένων (<https://postgis.net/>)
- Σε σύγκριση με την MySQL που επίσης είναι ανοικτού κώδικα σύστημα διαχείρισης RDBMS, υπερέχει σε απόδοση
- Η πιο δημοφιλής DBMS σύμφωνα με το StackOverflow Technology Survey 2023
- Υποστηρίζεται από την πλατφόρμα Heroku





<https://survey.stackoverflow.co/2023/#most-popular-technologies-database>



<https://survey.stackoverflow.co/2023/#most-popular-technologies-database>

PostgreSQL: Download as local database



Postgres.app

The easiest way to get started with PostgreSQL on the Mac

Introduction

Downloads

Documentation

GitHub

← 6294 Stars!

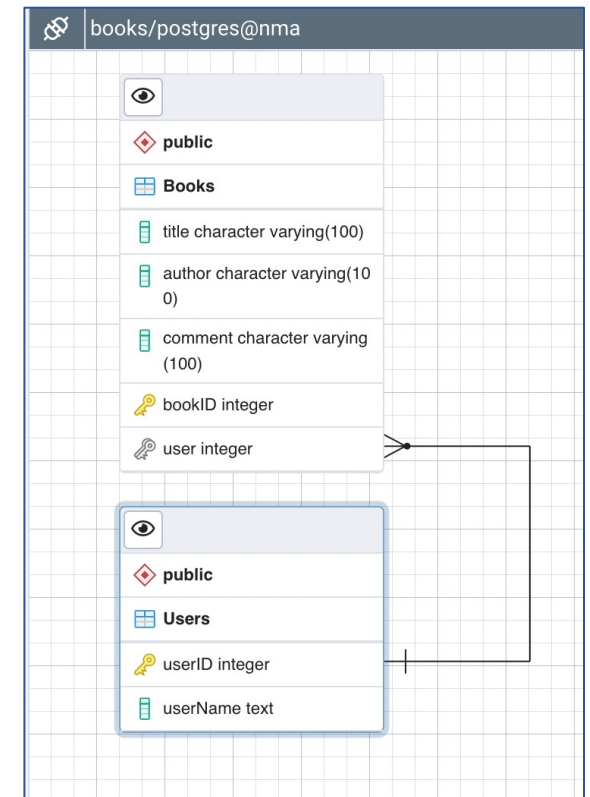
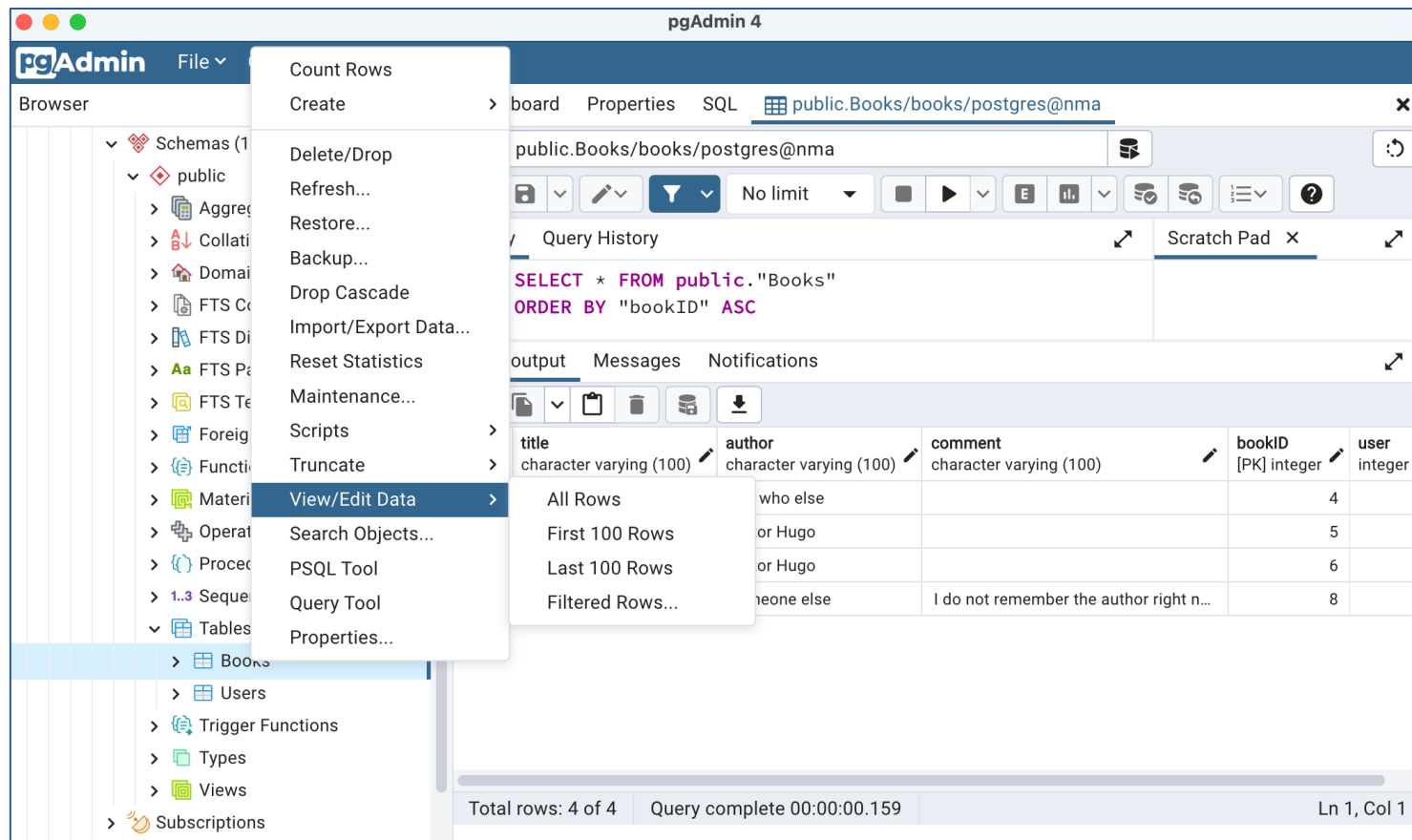
CLI Tools

Διαμορφώστε το \$PATH σας

- Η Postgres.app περιλαμβάνει εργαλεία γραμμής εντολών (CLI). Αν θέλετε να τα χρησιμοποιήσετε, πρέπει να ρυθμίσετε τη μεταβλητή \$PATH.
- Ο ευκολότερος τρόπος για να διαμορφώσετε το PATH σας είναι να εκτελέσετε την ακόλουθη εντολή:
- `sudo mkdir -p /etc/paths.d && echo /Applications/Postgres.app/Contents/Versions/latest/bin | sudo tee /etc/paths.d/postgresapp`
- Μην ξεχάσετε να κλείσετε το παράθυρο Terminal και να ανοίξετε ένα νέο για να τεθούν σε ισχύ οι αλλαγές.
- Μπορείτε να ελέγξετε αν η διαδρομή έχει ρυθμιστεί σωστά πληκτρολογώντας `$ which psql`.

PostgreSQL : Εργαλεία διαχείρισης

- Εκτός από το psql (γραμμή εντολών), μπορείτε να κατεβάσετε ένα από τα εργαλεία διαχείρισης, το πιο διαδεδομένο είναι το pgAdmin4



Σύνδεση στη βάση δεδομένων

```
const pool = new pg.Pool({ // τοπική σύνδεση
  user: ///username,
  host: 'localhost',
  database: 'books',
  password: /// password,
  port: 5432,
})
```

Η **δεξαμενή συνδέσεων** (connection pool) είναι μια προσωρινή μνήμη των συνδέσεων βάσης δεδομένων που επιτρέπει οι συνδέσεις να επαναχρησιμοποιούνται αντί να δημιουργούνται κάθε φορά που ζητείται μια σύνδεση.

```
const pool = new pg.Pool(); //οι παράμετροι ορίζονται ως
μεταβλητές περιβάλλοντος (βλέπε χρήση dotenv)
```

```
const pool = new pg.Pool({ //σύνδεση με URL
  connectionString: process.env.DATABASE_URL, //μεταβλητή
  περιβάλλοντος
  ssl: { rejectUnauthorized: false }
});
```

χρήση environment variable (dotenv)

npm install **dotenv**

```
import dotenv from "dotenv"
```

```
dotenv.config()
```

στο αρχείο .env που βρίσκεται στο φάκελο του πρότζεκτ μας αποθηκεύουμε τις μεταβλητές που επιθυμούμε ως environment variables, πχ

KEY=μιαπολυμεγάληλεξηπουδενμπορείκανειςναφανταστεί

έχουμε πρόσβαση στη μεταβλητή αυτή ως `process.env.KEY`

ΠΡΟΣΟΧΗ: στο .gitignore να βάλουμε το αρχείο .env

αρχείο .venv

```
PGHOST=localhost  
PGUSER= <username>  
PGDATABASE=books  
PGPASSWORD= <password>  
PGPORT=5432
```


findBook

```
import pg from "pg";
```

```
async function findBook(bookID, callback) {  
  const sql = `SELECT * FROM "Books" WHERE "bookID" = '${bookID}'`;  
  try {  
    const client = await connect();  
    const res = await client.query(sql)  
    await client.release()  
    callback(null, res.rows) // επιστρέφει array  
  }  
  catch (err) {  
    callback(err, null);  
  }  
}
```

```
export {getMyBooks, newBook, findBook, updateBook, deleteBook, insertUser,  
findUser};
```

Mongodb

δημιουργία online βάσης δεδομένων mongo

mongodb.com

- create a free account
- create a database
- create a starter cluster
(προσοχή **M0 sandbox**, free, shared Ram, 512 MB)
- Database access : create user
- Network access : IP white list -> allow access from anywhere

Create a Starter Cluster

Welcome to MongoDB Atlas! We've recommended some of our most popular options, but feel free to customize your cluster to your needs. For more information, check our [documentation](#).

Cloud Provider & Region AWS, Frankfurt (eu-central-1) ▼



★ Recommended region ⓘ

NORTH AMERICA	EUROPE	ASIA
 N. Virginia (us-east-1) ★	 Ireland (eu-west-1) ★	 Singapore (ap-southeast-1) ★
 Oregon (us-west-2) ★	 Frankfurt (eu-central-1) ★	 Mumbai (ap-south-1)
	AUSTRALIA	
	 Sydney (ap-southeast-2) ★	

Cluster Tier M0 Sandbox (Shared RAM, 512 MB Storage) ▼
Encrypted

Base hourly rate is for a MongoDB replica set with 3 data bearing servers.

Shared Clusters for development environments and low-traffic applications

Tier	RAM	Storage	vCPU	Base Price
✓ M0 Sandbox	Shared	512 MB	Shared	Free forever

M0 clusters are best for getting started, and are not suitable for production environments.

σύνδεση στη βάση δεδομένων

- cluster (αφού δημιουργηθεί) → Connect → Connect with mongo shell

1. **Select your operating system and download the mongo shell**

macOS Install via Homebrew `brew install mongodb/brew/mongodb-community-shell`

windows download mongo shell

https://downloads.mongodb.org/win32/mongodb-shell-win32-x86_64-2012plus-4.2.6.zip

2. **ADD** the download directory/bin to your \$PATH variable

3. **Run your connection string** in your command line

```
mongo "mongodb+srv://cluster0-brrcv.mongodb.net/test" --username ...
```

εισαγωγή στην mongo

τρέχουμε τον client mongo shell

```
mongo "mongodb+srv://cluster0-brrcv.mongodb.net/test" --username  
admin-nikos
```

MongoDB shell version v4.2.2

Enter password: ...

`show databases`

`use test` /// χρησιμοποιούμε και δημιουργούμε αν δεν υπάρχει την test

`db` /// μάς δείχνει την βάση δεδομένων που χρησιμοποιούμε

<https://docs.mongodb.com/manual/mongo/> → CRUD operations

mongo - πράξεις CRUD **insert, find**

```
db.collection.insertOne ( <document 1> );
```

```
db.collection.insertMany ( [ <document 1> , <document 2>, ... ] );
```

```
db.users.insertOne(
  {
    name: "sue",
    age: 26,
    status: "pending"
  }
)
```

← collection

← field: value
← field: value
← field: value } document

```
db.collection.find(query, projection)
```

```
db.users.find(
  { age: { $gt: 18 } },
  { name: 1, address: 1 }
).limit(5)
```

← collection
← query criteria
← projection
← cursor modifier

query operators

[\\$eq](#)
[\\$gt](#)
[\\$gte](#)
[\\$in](#)
[\\$lt](#)
[\\$lte](#)
[\\$ne](#)

[\\$and](#)
[\\$not](#)
[\\$nor](#)
[\\$or](#)

mongo - πράξεις CRUD **update, delete**

```
db.collection.updateOne(filter, update, options);  
db.collection.updateMany(filter, update, options);
```

```
db.users.updateMany(  
  { age: { $lt: 18 } },  
  { $set: { status: "reject" } }  
)
```



A diagram with three green arrows pointing left from text labels to the corresponding arguments in the MongoDB updateMany function call. The first arrow points from 'collection' to 'db.users'. The second arrow points from 'update filter' to '{ age: { \$lt: 18 } }'. The third arrow points from 'update action' to '{ \$set: { status: "reject" } }'.

```
db.collection.delete(filter);
```

```
db.users.deleteMany(  
  { status: "reject" }  
)
```



A diagram with two green arrows pointing left from text labels to the arguments in the MongoDB deleteMany function call. The first arrow points from 'collection' to 'db.users'. The second arrow points from 'delete filter' to '{ status: "reject" }'.

σύνδεση mongo server με την node.js

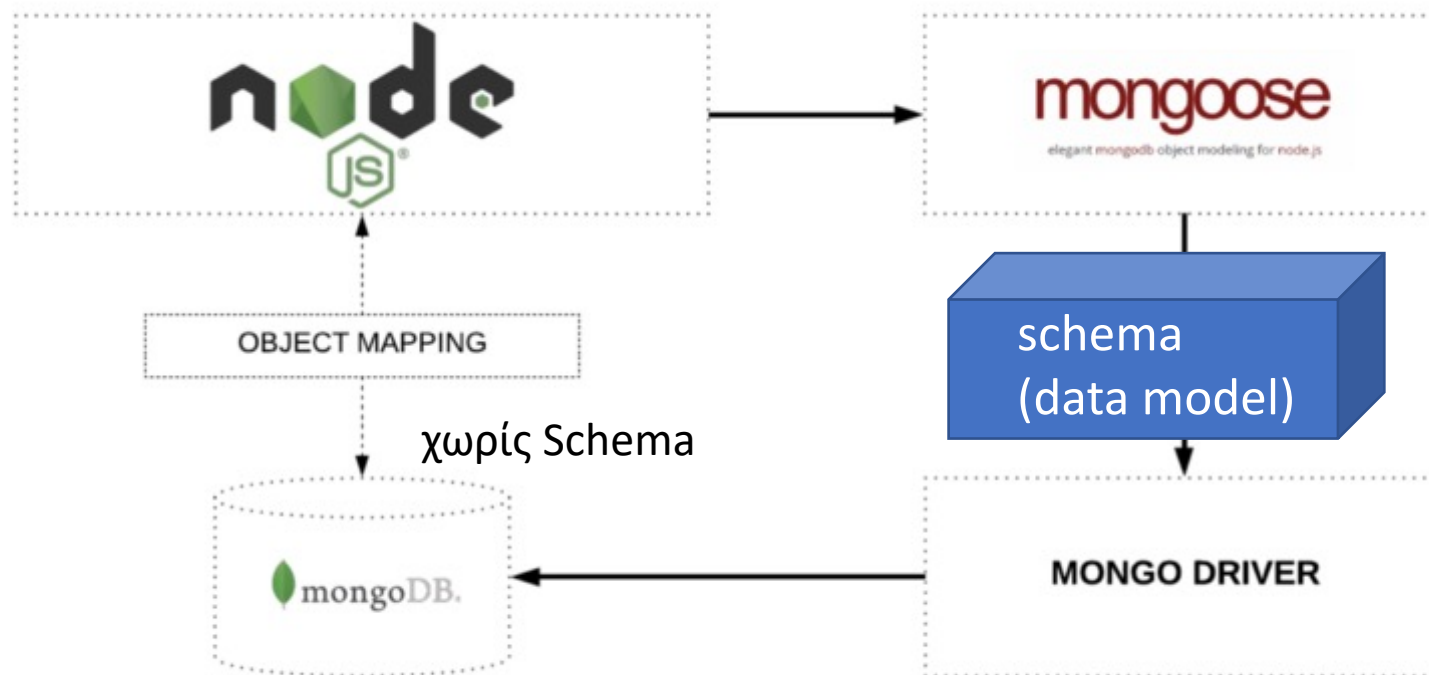
Χρειάζεται η εγκατάσταση και χρήση του **mongoose** (mongoosejs.com) ενός Object-Document Mapper (ODM) που απαιτεί τη χρήση model

```
const mongoose = require('mongoose');
mongoose.connect('mongodb://localhost:27017/testDB',
  {useNewUrlParser: true, useUnifiedTopology: true});

const db = mongoose.connection

db.on('error', console.error.bind(console, 'connection error:'));
db.once('open', function() { // μήνυμα επιτυχούς σύνδεσης});
```


σύνδεση node.js με mongo μέσω mongoose



mongoose; δημιουργία schema, model, collection

Με την mongoose τα δεδομένα μιας συλλογής πρέπει να είναι σύμφωνα με ένα σχήμα (Schema) που πρέπει να το ορίσουμε από πριν

```
const kittySchema = new mongoose.Schema({  
  name: String });
```

Στη συνέχεια με βάση το σχήμα αυτό ορίζουμε ένα μοντέλο που χρησιμοποιεί την default connection

```
const Kitten = mongoose.model('Kitten', kittySchema, 'kitten');
```

model name, schema name, collection name

mongoose; εισαγωγή documents

Έστω ότι έχουμε ορίσει το μοντέλο του προηγούμενου παραδείγματος

```
const Kitten = mongoose.model('Kitten', kittySchema, 'kitten');
```

model name, schema name, collection name

Δύο τρόποι για να εισάγουμε documents στην συλλογή kitten

(α) μπορούμε να δημιουργήσουμε αντικείμενα και να τα αποθηκεύσουμε

```
const fluffy = new Kitten({ name: 'fluffy' });  
fluffy.save([callback]);
```

(β) μπορούμε να καλέσουμε τη μέθοδο insertMany() του model Kitten

```
kitten.insertMany(  
  [ { "name" : "Fluffy"}, { "name" : "Mini" } ],  
  (err, res)=> { if (err) {  
    console.log (err)}  
    else { console.log(res) } });
```

mongoose; αναζήτηση documents

Έστω ότι έχουμε ορίσει το μοντέλο του προηγούμενου παραδείγματος

```
const Kitten = mongoose.model('Kitten', kittySchema, 'kitten');
```

model name, schema name, collection name

Για να αναζητήσουμε documents στην συλλογή kitten

```
Kitten.find({name: "Mini"}, (err, res) => {  
  // error handling and result processing } );
```

Model.find(). - Model.findById(). - Model.findByIdAndDelete().
Model.findByIdAndRemove(). - Model.findByIdAndUpdate(). - Model.findOne().
Model.findOneAndDelete().
Model.findOneAndRemove(). - Model.findOneAndReplace(). -

mongoose; τροποποίηση documents

Έστω ότι έχουμε ορίσει το μοντέλο του προηγούμενου παραδείγματος

```
const Kitten = mongoose.model('Kitten', kittySchema, 'kitten');
```

model name, schema name, collection name

Για να τροποποιήσουμε ένα document στην συλλογή kitten

```
Kitten.updateOne({name: "Mini"}, {$set: {name: "Micky"}} , (err, res) => {  
    // error handling and result processing } );
```

Model.update(). - Model.updateMany(). -
Model.updateOne()

mongoose; διαγραφή documents

Έστω ότι έχουμε ορίσει το μοντέλο του προηγούμενου παραδείγματος

```
const Kitten = mongoose.model('Kitten', kittySchema, 'kitten');
```

model name, schema name, collection name

Για να διαγράψουμε documents στην συλλογή kitten deleteOne(συνθήκη, callback)

```
Kitten.deleteOne({name: "Mini"}, (err) => {  
  // error handling } );
```

Model.deleteMany()

Model.deleteOne()

mongoose: διασύνδεση documents

Έστω ότι θέλουμε σε ένα νέο schema **personSchema** να ενσωματώσουμε το **KittySchema** που έχουμε ήδη δημιουργήσει

```
const kittySchema = new mongoose.Schema({  
  name: String });  
  
const personSchema = new mongoose.Schema({  
  name: String,  
  age: Number,  
  favouriteCat: kittySchema});
```

Τα βιβλία της παρέας mongoose

```
// MONGOOSE schemata and model definition
```

```
const bookSchema = new mongoose.Schema(  
  {title: String,  
   author: String,  
   comment: String,  
  });
```

```
const userSchema = new mongoose.Schema(  
  {userName: String,  
   books: [bookSchema]});
```

```
const Book = await new mongoose.model("Book", bookSchema, "books")  
const User = await new mongoose.model("User", userSchema, "users")
```

```
_id: ObjectId('645dc8a2c9a52e1bcea64a7a')  
userName: "nikos"  
▼ books: Array  
  ▼ 0: Object  
    title: "Δον Κιχώτης"  
    author: "Θερβάντες"  
    comment: ""  
    _id: ObjectId('645dd76778d1762da9339068')  
  ▼ 1: Object  
    title: "Ποιήματα"  
    author: "Κ. Καβάφης"  
    comment: "εξαιρετικά"  
    _id: ObjectId('645dd77478d1762da933906d')  
__v: 0
```


Μετατροπή από sqlite3 σε mongoose

controller

```
import * as model from '../..../model/model_lite.mjs';

function createBook(req, res){
  const userID = req.session.userID;
  console.log(userID);
  const newBook = {"title":req.body.title, "author":req.body.author,
                  "comment":req.body.comment, "user": req.session.userID}
  model.newBook(newBook, (err, data)=> {
    if (err)
      return console.error(err.message);
    else
      res.redirect("/books");
  });
}

export default createBook
```

Μετατροπή από sqlite3 σε mongoose

controller

```
import * as model from '../..../model/model_mongo.mjs';

async function createBook(req, res){
  const userID = req.session.userID;
  console.log(userID);
  const newBook = {"title":req.body.title, "author":req.body.author,
                  "comment":req.body.comment}

  try {
    await model.newBook(newBook, userID);
    res.redirect("/books");
  } catch(err){
    return console.error(err.message);
  }
}

export default createBook
```

Μετατροπή από sqlite3 σε mongoose

model

```
const newBook = (book, callback) => {  
  const sql = `INSERT INTO Books (title, author, comment, user)  
              VALUES (?, ?, ?, ?)`;  
  const db = new sqlite3.Database(db_name);  
  db.run(sql, [book.title, book.author, book.comment, book.user],  
    (err, result) => {  
      db.close();  
      if (err) {  
        callback(err, null);  
      } else {  
        callback(null, result);  
      }  
    });  
}
```

Η δημιουργία εφαρμογής στην πλατφόρμα
Heroku (application as a service)



heroku / deploying the application

- Η **Heroku** είναι μια πλατφόρμα ως υπηρεσία (**PaaS**) που επιτρέπει στους προγραμματιστές να δημιουργούν, να εκτελούν και να λειτουργούν εφαρμογές εξ ολοκλήρου στο νέφος.
- Μια εφαρμογή εκτελείται σε ένα **dyno** (ένα κουτί Linux).
- Η Heroku χρησιμοποιεί το **AWS (Amazon Web Services)**.
- Η **Heroku** υποστηρίζει εφαρμογές γραμμένες σε: Node.js, Python, Java, PHP, Go κ.λπ.



heroku / δωρεάν λογαριασμός

Το dyno κοιμάται μετά από 30 λεπτά αδράνειας

Ένας επαληθευμένος λογαριασμός συνοδεύεται από 1000 dyno-ώρες (CPU) ανά μήνα

Μπορούν να δημιουργηθούν προσαρμοσμένα από τον χρήστη domains όπως **https://application-name.herokuapp.com**

Η θύρα καθορίζεται από το περιβάλλον

```
// Εκκίνηση του εξυπηρετητή
const PORT = process.env.PORT || 3003
app.listen(PORT, () => {
  console.log(`Συνδεθείτε στη σελίδα:
  http://localhost:${PORT}`);
});
```



heroku / deploying the application

- Δημιουργήστε λογαριασμό Heroku
- Κατεβάστε και εγκαταστήστε το [Heroku CLI](#).
- Συνδεθείτε στον λογαριασμό σας και ακολουθήστε τις οδηγίες ώστε να δημιουργήσετε ένα δημόσιο κλειδί SSH public key.
- `$ heroku login`
- Τροποποιήστε την εφαρμογή σας για να ικανοποιεί τις απαιτήσεις του Heroku
- Ανεβάστε την εφαρμογή στο Heroku χρησιμοποιώντας το Git.
 - `$ git add .`
 - `$ git commit -am "make it better"`
 - `$ heroku git:clone -a mybooks-app`
 - `$ git push heroku master`

Κάνουμε αίτημα για δημιουργία βδ postgresQL και τη συνδέουμε με την εφαρμογή που έχουμε δημιουργήσει



Heroku Postgres
Hobby Dev - Free

→



mybooks-app
Personal apps

In summary this will provision **Heroku Postgres** on your personal **mybooks-app** application, using the **Hobby Dev** plan (Free).

[Change destination app](#)

By submitting this order form, you agree that the Add-on is governed by the applicable provider's terms of use, and the Heroku Services are governed by the [Salesforce Master Subscription Agreement](#), unless (except for free customers) you have entered into a written Master Subscription Agreement executed by SFDC for the Heroku Services as referenced in the Documentation.

[Submit Order Form](#)

 DATA

Databases >  postgresql-cubed-62592

SERVICE heroku-postgresql PLAN hobby-dev BILLING APP  mybooks-app

Overview Durability Settings Dataclips

HEALTH

 Available

REGION Europe PRIMARY Yes VERSION 14.2 CREATED a minute ago MAINTENANCE Unsupported ⓘ

ROLLBACK Unsupported ⓘ

UTILIZATION

0 of 20 CONNECTIONS 0 of 10,000 ROWS  IN COMPLIANCE 8.5 MB DATA SIZE

0 TABLES

heroku.com Blogs Careers Documentation [Support](#) Terms of Service Privacy Cookies © 2022 Salesforce.com

heroku pg:psql ... create table , etc..

```
mybooks-app::DATABASE=> select * from "Books";
```

```
  title | author | comment | bookID | user
```

```
-----+-----+-----+-----+-----
```

```
(0 rows)
```

```
mybooks-app::DATABASE=> select * from "Users";
```

```
  userID | userName
```

```
-----+-----
```

```
(0 rows)
```

Δημιουργία βδ από export της τοπικής βδ

```
CREATE TABLE "Books" (  
  "title" character varying(100) NOT NULL,  
  "author" character varying(100),  
  "comment" character varying(100),  
  "bookID" integer NOT NULL,  
  "user" integer  
);
```

```
CREATE TABLE "Users" (  
  "userID" integer NOT NULL,  
  "userName" text  
);
```

```
ALTER TABLE "Users" ALTER COLUMN "userID" ADD  
GENERATED ALWAYS AS IDENTITY (  
  SEQUENCE NAME public."Users_userID_seq"  
  START WITH 1  
  INCREMENT BY 1  
  NO MINVALUE
```

```
  NO MAXVALUE  
  CACHE 1  
);
```

```
ALTER TABLE "Books" ALTER COLUMN "bookID" ADD  
GENERATED ALWAYS AS IDENTITY (  
  SEQUENCE NAME public."books_bookID_seq"  
  START WITH 1  
  INCREMENT BY 1  
  NO MINVALUE  
  NO MAXVALUE  
  CACHE 1  
);
```

```
ALTER TABLE ONLY "Users" ADD CONSTRAINT pk_1 PRIMARY  
KEY ("userID");
```

```
ALTER TABLE ONLY "Books" ADD CONSTRAINT "pk_bookID"  
PRIMARY KEY ("bookID");
```

Σύνδεση της βδ με την εφαρμογή Heroku

Εισάγουμε το URI της Database credentials στο αρχείο .env

```
DATABASE_URL: postgres://paxdhrppercbkb:440e6775c721 ...
```

Κάνουμε τη σύνδεση με χρήστη του URI

```
const pool = new pg.Pool({  
  connectionString: process.env.DATABASE_URL,  
  ssl: {  
    rejectUnauthorized: false  
  }  
});
```

Η εφαρμογή online

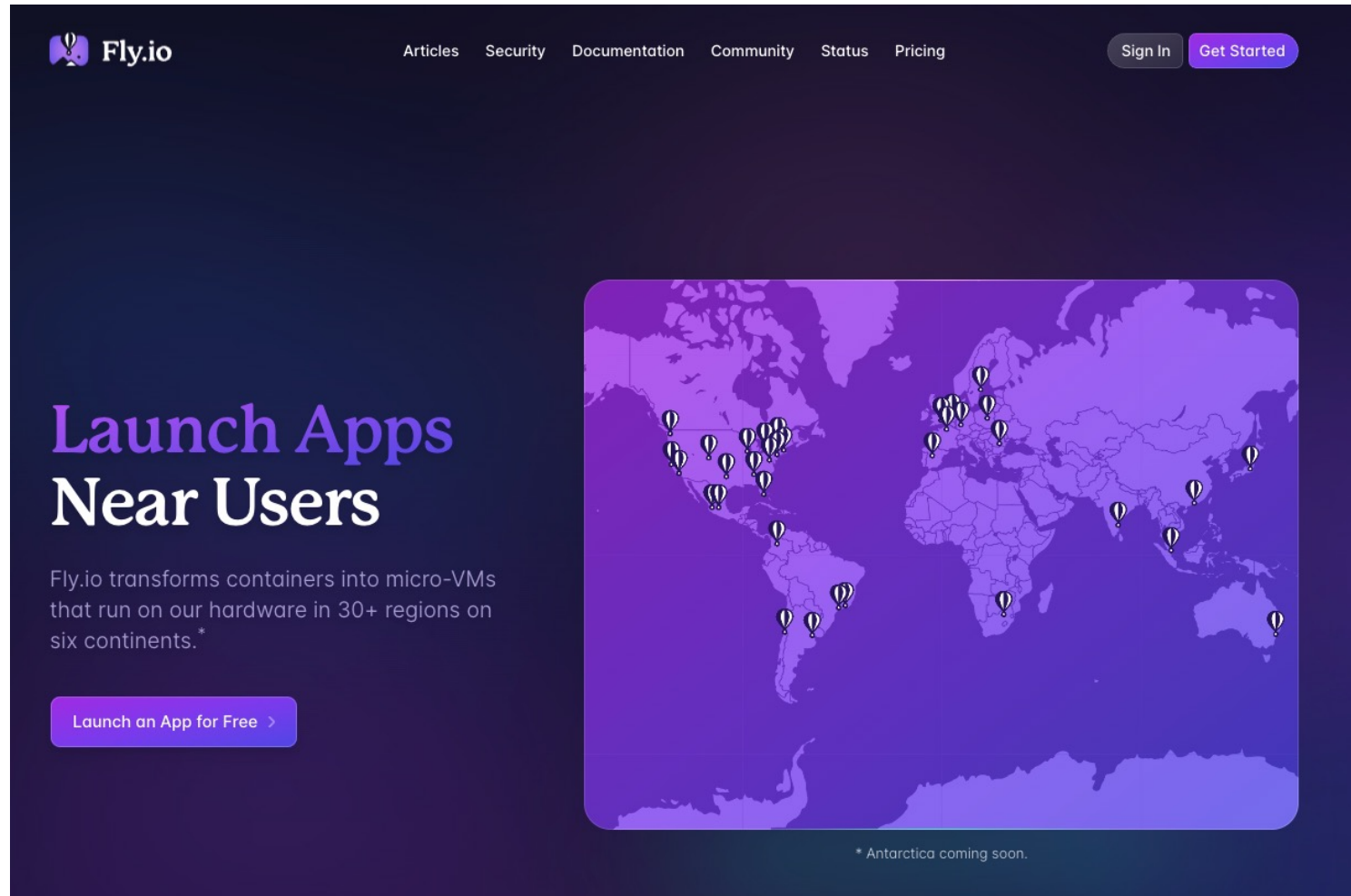
 myBooks βιβλία... σχετικά..

τα βιβλία μου - Nikos (1)

Τίτλος	Συγγραφείς	Σχόλια	Προσθήκη
Εισαγωγή στην Python	N. Αβούρης	Μαζί με την Python μαθαίνουμε για τους υπολογιστές	Αλλαγή Διαγραφή

© ECE_ΓΚ802 - myBooks

Ανάπτυξη στην υπηρεσία fly.io



The screenshot shows the Fly.io website homepage. The header features the Fly.io logo on the left and navigation links (Articles, Security, Documentation, Community, Status, Pricing) and buttons (Sign In, Get Started) on the right. The main content area has a dark blue background. On the left, the text 'Launch Apps Near Users' is displayed in a large, bold, light blue font. Below this, a paragraph explains that Fly.io transforms containers into micro-VMs running on hardware in 30+ regions across six continents. A button labeled 'Launch an App for Free >' is positioned below the text. On the right, a world map is shown with numerous hot air balloon icons indicating global deployment regions. At the bottom right, a small note states '* Antarctica coming soon.'

Fly.io

Articles Security Documentation Community Status Pricing

Sign In Get Started

Launch Apps Near Users

Fly.io transforms containers into micro-VMs that run on our hardware in 30+ regions on six continents.*

Launch an App for Free >

* Antarctica coming soon.

Εγκατάσταση flyctl (command tool)

Windows

```
powershell -Command "iwr https://fly.io/install.ps1 -useb | iex"
```

Linux

```
curl -L https://fly.io/install.sh | sh
```

Macos

```
brew install flyctl
```

<https://medium.com/@kjolee/deploying-a-node-js-app-on-fly-io-3d511bbf34ac>

login - launch app

```
fly auth login // login me github
```

```
fly launch //
```

Creating app in /home/user/projects/my-cool-app

Scanning source code

Detected a NodeJS app

```
? Choose a region for deployment: [Use arrows to move, type to filter]
Madrid, Spain (mad)
Miami, Florida (US) (mia)
Tokyo, Japan (nrt)
Chicago, Illinois (US) (ord)
Bucharest, Romania (otp)
Phoenix, Arizona (US) (phx)
Querétaro, Mexico (qro)
Santiago, Chile (scl)
Seattle, Washington (US) (sea)
Singapore, Singapore (sin)
San Jose, California (US) (sjc)
Sydney, Australia (syd)
Warsaw, Poland (waw)
Montreal, Canada (yul)
> Toronto, Canada (yyz)
```

επιλογή κοντινότερου server

δημιουργία βάσης δεδομένων

```
? Would you like to set up a Postgresql database now? Yes
? Select configuration: [Use arrows to move, type to filter]
> Development - Single node, 1x shared CPU, 256MB RAM, 1GB disk
  Production (High Availability) - 3 nodes, 2x shared CPUs, 4GB RAM, 40GB disk
  Production (High Availability) - 3 nodes, 4x shared CPUs, 8GB RAM, 80GB disk
  Specify custom configuration
```

επιλογή Development configuration

Postgres cluster (your database name) created

Username: (assigned username)

Password: (assigned password)

Hostname: (your database name).internal

Flycast: (some address)

Proxy port: (some port number)

Postgres port: (some other port number)

Connection string: (your connection string)

αποθηκεύστε τα στοιχεία και κωδικούς
σε ασφαλές μέρος

δημιουργία σχήματος βάσης δεδομένων

- Χρειάζεται να δημιουργήσετε το σχήμα της βάσης δεδομένων

```
fly pg connect -a (your database name)
\l
```

List of databases							
Name	Owner	Encoding	Collate	Ctype	ICU Locale	Locale Provider	Access privileges
postgres	postgres	UTF8	en_US.utf8	en_US.utf8		libc	
repmgr	repmgr	UTF8	en_US.utf8	en_US.utf8		libc	
template0	postgres	UTF8	en_US.utf8	en_US.utf8		libc	=c/postgres +
template1	postgres	UTF8	en_US.utf8	en_US.utf8		libc	postgres=CTc/postgres +
withered_wind_7171	postgres	UTF8	en_US.utf8	en_US.utf8		libc	=c/postgres +
postgres=CTc/postgres							
(5 rows)							

\c βάση δεδομένων

\i schema.sql ή εντολές δημιουργίας πινάκων

fly.toml

Οι μεταβλητές που έχουμε
συνήθως στο process.env
θα πρέπει να οριστούν στο
αρχείο fly.toml περιοχή
[env]

toml = Tom's Obvious,
Minimal Language (github
configuration file format)

[app]: καθορίζει το όνομα της εφαρμογής σας όπως αναγνωρίζεται στο Fly.io.

[app] **name** = "your-app-name"

[build]: Λεπτομέρειες σχετικά με τον τρόπο κατασκευής της εφαρμογής σας, συμπεριλαμβανομένης της τοποθεσίας του αρχείου Docker ή των εικόνων δημιουργού

[build] **image** = "registry.example.com/org/image:tag"

[deploy]: Διαμόρφωση που επηρεάζει τη διαδικασία ανάπτυξης, όπως οι εντολές release

[deploy] **release_command** = "bundle exec rake db:migrate"

[env]: Μεταβλητές περιβάλλοντος

[env] **PORT** = "8080"

[services]: Καθορίζει υπηρεσίες δικτύου, θύρες και χειριστές. Αυτό είναι ζωτικής σημασίας για τη ρύθμιση του τρόπου με τον οποίο η εφαρμογή εκθέτει τις υπηρεσίες της και πώς χειρίζεται την εισερχόμενη κυκλοφορία

[[services]]

internal_port = 8080

protocol = "tcp"

[[services.http_checks]]

interval = 10000

timeout = 2000

method = "get"

path = "/health"

[[services.ports]] **port** = 80

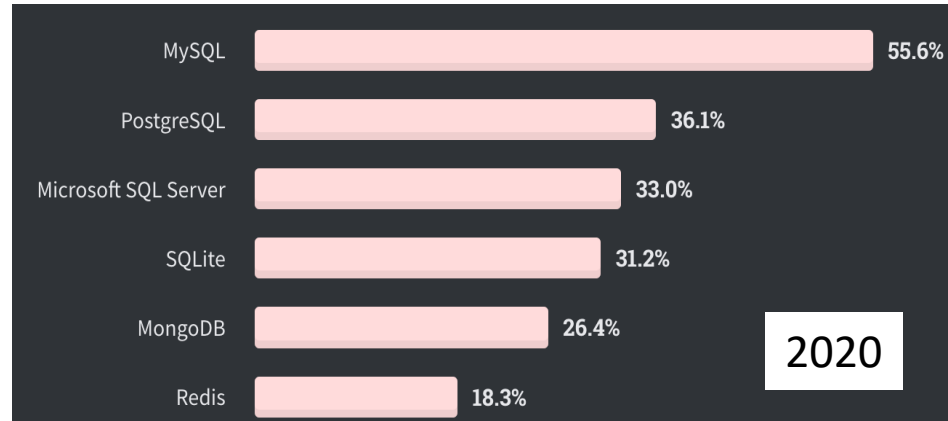
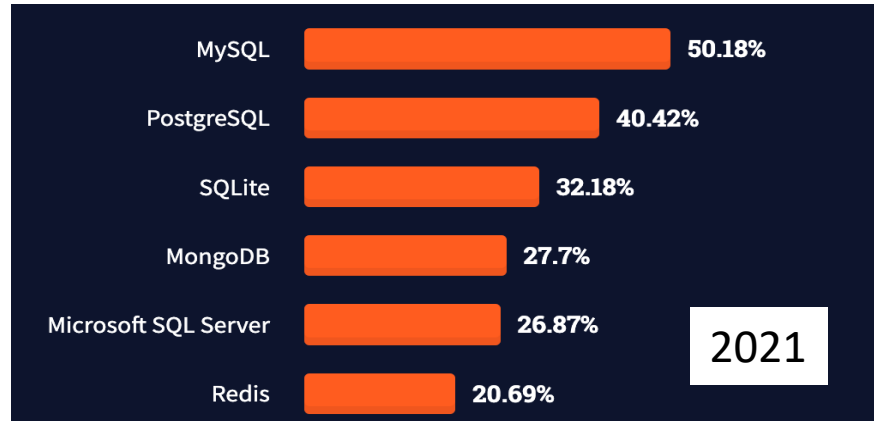
force_https = true

[[services.tcp_checks]] **interval** = 10000 **timeout** = 2000

MySQL

MySQL

- Πολύ δημοφιλές σχεσιακό Σύστημα διαχείρισης βάσεων δεδομένων (RDBMS) ανοιχτού κώδικα.
- Η MySQL χρησιμοποιείται από πολλές διαδικτυακές εφαρμογές που βασίζονται σε βάσεις δεδομένων, όπως το Drupal, το Joomla, το phpBB και το WordPress, Facebook, το Flickr, το MediaWiki, το Twitter και το YouTube.
- Δεν υποστηρίζεται από τη Heroku (μόνο Redis, PostgreSQL)



Εναλλακτική λύση:

<https://remotemysql.com/>

Αν ξεπεράσετε το πρόβλημα με την επιθετική διαφήμιση και surveys

avouris@g.upatras.gr

RemoteMySQL - Dashboard

Databases

All your databases are listed below. At the bottom is a button to create new databases. If requested, you may need to complete a quick survey to help support keeping this service free. Ensure you complete it correctly as responses are checked for quality and may get rejected.

USERNAME	SIZE	CREATED DATE	ACTION
57sc1S3DN6	0.08 MB	2020-04-29 13:38:48	ACTION Statistics Permissions Reset Password Delete phpMyAdmin

```
{ "connectionLimit": 10,
  "host": "remotemysql.com",
  "user": "xxxxxxxx",
  "password": "yyyyyyyyyy",
  "database": "dddddddddd",
  "port": "3306" }
```

(1) reset password
(2) με χρήση του
username/password →
phpMyAdmin

<https://www.mysqltutorial.org/mysql-nodejs/>

Σύνδεση στην MySQL

```
const connection = mysql.createConnection({  
  host: 'localhost',  
  user: 'username',  
  password: 'password',  
  database: 'my-database' });
```

```
connection.connect(function(err) {  
  if (err) {  
    return console.error('error: ' + err.message);  
  }  
  console.log('Connected to MySQL server.');
```

Pool (δεξαμενή συνδέσεων)

```
const pool = mysql.createPool({  
  connectionLimit: 5,  
  host: 'host-url',  
  user: 'user-name',  
  password: 'password',  
  database: 'my-db' });
```

Δημιουργία δεξαμενής
συνδέσεων

Ανάκτηση σύνδεσης από pool

```
pool.getConnection(function(err, connection) {  
  // execute query // ... });
```

```
pool.end(function(err) {  
  if (err) {  
    return console.log(err.message);  
  }  
  // close all connections });
```

Κλείσιμο όλων των συνδέσεων
του pool

Εκτέλεση queries `connection.query( sql, data, callback)`

```
import mysql from 'mysql';
import config from './config.js';
const connection = mysql.createConnection(config);
let sql = `SELECT * FROM table WHERE id=?`;
connection.query(sql, ['1'], (error, results, fields) => {
    if (error) {
        return console.error(error.message);
    }
    console.log(results);
});
connection.end();
```

Για προστασία από SQL injection

```
let sql = `SELECT * FROM todos WHERE id = ` + mysql.escape(id);
```

Εκτέλεση queries με promises

Using promises allows us to process the result of the query and not create a call-back hell

```
const mysql = require('mysql');
const util = require('util');
const config = require('./config.js');

let connection = mysql.createConnection(config);
let sql = `SELECT * FROM table WHERE id=?`;
connection.query(sql, ['1'], (error, results, fields) => {
    if (error) {
        return console.error(error.message);
    }
    console.log(results);
});
connection.end();
```

Εκτέλεση queries

- Η μέθοδος `mysql.query` δέχεται, `sql-string`, έναν πίνακα από ορίσματα και μια συνάρτηση επανάκλησης (`callback`) που καλείται για να χειριστεί τα αποτελέσματα
- Αν θέλουμε να περάσουμε τα αποτελέσματα σε μια άλλη συνάρτηση, θα μπορούσαμε να χρησιμοποιήσουμε τη σύνταξη `async/await promises`.

παράδειγμα

```
fs.readFile("remotemysqlConfig.json",{encoding: 'utf8'}, (err, jsonString)=>{
  // we read the configuration file for connection to the remotemysql data base
  if(err) throw console.log('no remotemysqlAuth.json file', err);
  else {
    // we create a pool of connections
    const pool = mysql.createPool(JSON.parse(jsonString));
    try {
      // we execute a query in the data base
      let sql = 'SELECT ... WHERE ?';
      let data = [5, 5]
      pool.query(sql, data, (error, result) => {
        // for the results, we print the results to the console
        if(error) throw error;
        console.log(result);
      })
    } catch (error) {
      console.log('Error executing sql query:', error)
      return false;
    }
  }
});
```

Επανάληψη SQL μέσα από ένα παράδειγμα

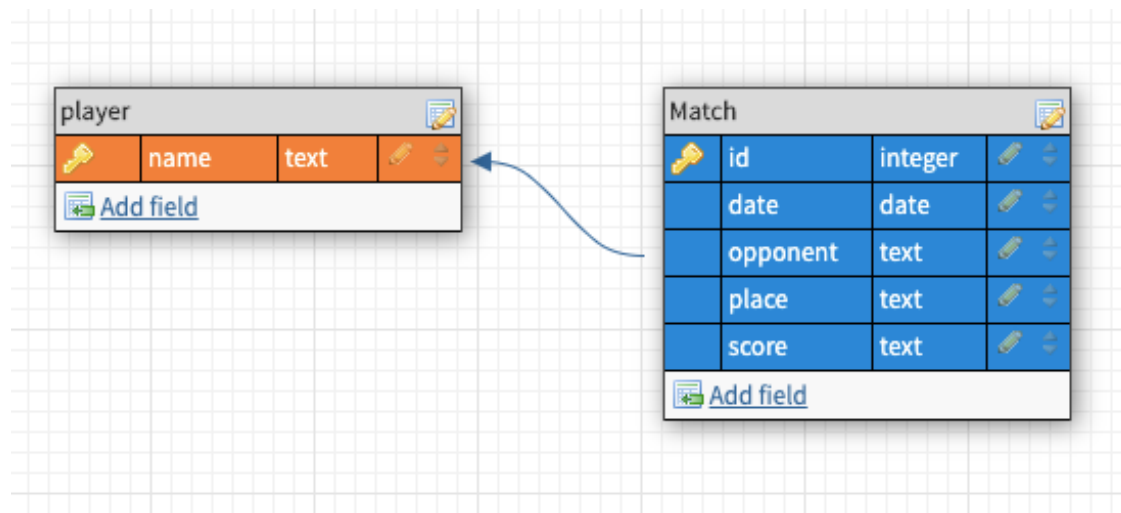
myTennisApp

Είμαι ένας παίκτης τένις και ζητάω να σχεδιάσετε μια εφαρμογή που καταγράφει το σκορ των αγώνων που δίνω

πχ είμαι ο Στέφανος Τσιτσιπάς και
θέλω να θυμάμαι ότι στις
16.11.2019 σε αγώνα εναντίον του
Roger Federer στο ATP WT Finals
νίκησα με 6-3, 6-4

 Stefanos Tsitsipas Greece	
CHAMPIONSHIPS	ITALIAN OPEN
LAVAR CUP	SWIS
Sat, Nov 16, 2019	
Semi-final · Center Court	
Final >	
 6 S. Tsitsipas	6 6 ◀
 3 R. Federer	3 4
ATP World Tour Finals scores & schedule	

σχεδιάζω τη βάση δεδομένων της myTennisApp



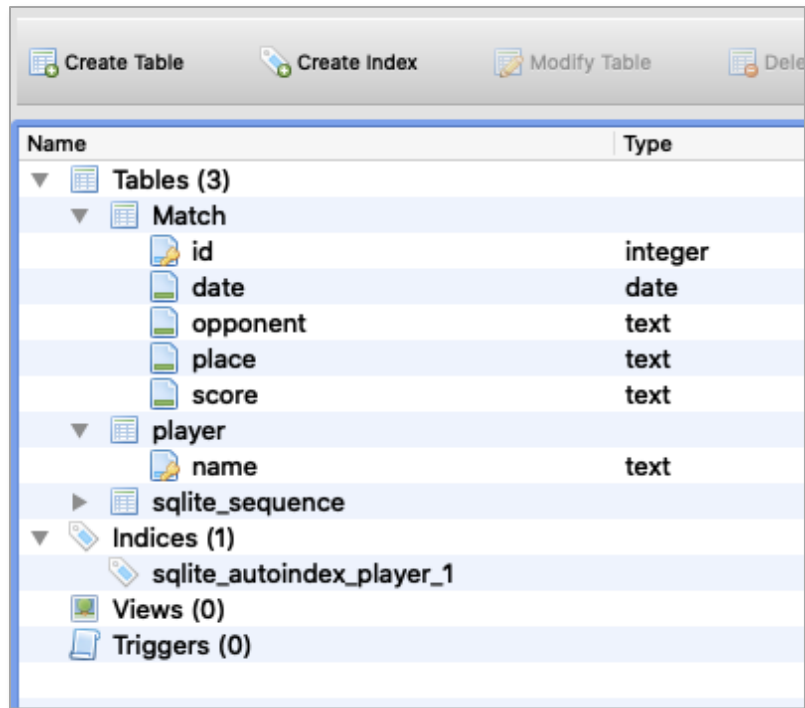
```
CREATE TABLE player (  
    name text PRIMARY KEY );
```

```
CREATE TABLE Match (  
    id integer PRIMARY KEY AUTOINCREMENT,  
    date date,  
    opponent text,  
    place text,  
    score text );
```

dbdesigner.net

create tabel/ add field/ save schema, export sql (sqlite)

δημιουργώ τη βάση δεδομένων my-tennis-app



CRUD λειτουργίες:

- CREATE
- READ
- UPADATE
- DELETE

DB Browser for SQLite /. → import sql file

CRUD λειτουργίες στη βάση δεδομένων: INSERT

```
INSERT INTO player VALUES ("Federer");
INSERT INTO player VALUES ("Djokovic");
INSERT INTO player VALUES ("Nadal");
INSERT INTO match VALUES ("Federer");
INSERT INTO match(date, opponent, score)
VALUES ("2019-11-19", "Federer", "6-3, 6-4");
INSERT INTO match(date, opponent, score)
VALUES ("2019-11-15", "Nadal", "6-7, 6-4, 7-5");
```

name	
	Filter
1	Federer
2	Djokovic
3	Nadal

	id	date	opponent	place	score
		Filter	Filter	Filter	Filter
1	1	2019-11-19	Federer	NULL	6-3, 6-4
2	2	2019-11-15	Nadal	NULL	6-7, 6-4, 7-5

CRUD λειτουργίες στη βάση δεδομένων: **UPDATE**

```
UPDATE match  
  SET place = "ATP WT Finals"  
 WHERE id = 1;
```

id		date	opponent	place	score
Filter		Filter	Filter	Filter	Filter
1	1	2019-11-19	Federer	ATP WT Finals	6-3, 6-4
2	2	2019-11-15	Nadal	NULL	6-7, 6-4, 7-5

CRUD λειτουργίες στη βάση δεδομένων: DELETE

```
DELETE FROM player  
WHERE name = "Djokovic";
```

name	
Filter	
1	Federer
2	Djokovic
3	Nadal

CRUD λειτουργίες στη βάση δεδομένων: **SELECT**

```
SELECT opponent, score  
FROM match  
WHERE 1;
```

φέρε τα **γνωρίσματα**
από τον **πίνακα**
που ικανοποιούν τη **συνθήκη**

	opponent	score
1	Federer	6-3, 6-4
2	Nadal	6-7, 6-4, 7-5